Yaqeen — Line-by-Line Explanation (XAMPP + axios integration)

Covers **everything** built across the two sessions: - **Session 1 — WRITE side:** forms send data INTO the database (INSERT / UPDATE). - **Session 2 — READ side:** pages pull data OUT of the database (SELECT) and show it.

PART 0 — The mental model

Three programs running at once:

Program	Where	Job
React (Vite)	browser, `http://localhost:5173`	the screens the user sees
Apache + PHP	`http://localhost` (port 80)	runs `.php` files in `D:\xampp\htdocs\yaqeen-backend\`
MySQL	port 3306	stores the data in the `yaqeen_web_project` database

axios is the messenger that carries data between React and PHP over HTTP.

Two directions:

```
WRITE:  form  → axios.post → PHP → INSERT/UPDATE → MySQL
READ :  page  → axios.get  → PHP → SELECT        → MySQL → JSON back → screen
```

PART 1 — connection.php (shared by every PHP file)

```php
<?php
    $connect = mysqli_connect("localhost", "root", "", "yaqeen_web_project");

    if (!$connect) {
        die("Connection failed: " . mysqli_connect_error());
    }
?>
```

- `<?php` — start PHP code.
- `mysqli_connect(host, user, password, database)` — opens a live connection to MySQL.
- `"localhost"` = MySQL is on this same PC.
- `"root"` = the database username.
- `""` = empty password (XAMPP's default for root).

- `"yaqeen_web_project"` = which database to use.
- The result (a connection handle) is saved in the variable `$connect`.
- `if (!$connect)` — `!` means "not". So: "if there is NO connection".
- `die(...)` — stop the whole script immediately and print the message.
- `mysqli_connect_error()` — the reason the connection failed.
- `.` in PHP = glue two strings together (like `+` for strings in JS).
- Every other PHP file runs `include("connection.php")`, which means "paste this file's code here", so they all get the same `$connect`.

PART 2 — The CORS + OPTIONS block (top of EVERY endpoint)

Every endpoint starts with these 9 lines. Explained once here; identical everywhere.

```
header("Access-Control-Allow-Origin: *");
header("Access-Control-Allow-Methods: POST, GET, OPTIONS");
header("Access-Control-Allow-Headers: Content-Type");

if ($_SERVER['REQUEST_METHOD'] == 'OPTIONS') {
    http_response_code(200);
    exit();
}
```

- React (port 5173) and PHP (port 80) are **different origins**. By default browsers BLOCK a page from calling a different origin (security rule called CORS).
- `header("Access-Control-Allow-Origin: *")` — PHP tells the browser "I allow calls from anyone (`*`)".
- `Allow-Methods` — which HTTP verbs are allowed (POST/GET/OPTIONS).
- `Allow-Headers: Content-Type` — allow the header that says "I'm sending JSON".
- `$_SERVER['REQUEST_METHOD']` — what kind of request came in (GET, POST, OPTIONS...).
- Before a real POST, the browser secretly sends a **preflight** `OPTIONS` request asking "am I allowed?".
- `if (... == 'OPTIONS')` — if this is that test request:
- `http_response_code(200)` — answer "200 OK = yes allowed".
- `exit()` — stop here (nothing else to do for a test request).
- Without this block, the real request never happens and you get a CORS error in the console.

PART 3 — WRITE endpoints (Session 1)

3.1 register_processing.php

After the CORS block:

```
include("connection.php");
```

- Pastes in `connection.php` → now we have `$connect`.

```
$rawData = file_get_contents("php://input");
$data = json_decode($rawData);
```

- `axios.post` sends the form as a **JSON string** in the request body.
- `file_get_contents("php://input")` — read that raw body text into `$rawData` (e.g. `{"email":"a@b.com","password":"123"}`).
- `json_decode($rawData)` — turn that JSON text into a PHP object stored in `$data`. Now `$data->email` works.
- `->` in PHP = read a property of an object (like `.` in JS: `data.email`).

```
if (isset($data->email) && isset($data->password)) {
```

- `isset(...)` — "does this field exist and is not empty/null?".
- `&&` — AND. So: only proceed if BOTH email and password were sent. A safety check.

```
$accountType = $data->accountType;
$fullName    = $data->fullName;
$email       = $data->email;
// ...the rest of the fields...
$password    = $data->password;
```

- Copy each value out of `$data` into a short variable. Just for readability in the query below.

```
$businessName = isset($data->businessName) ? $data->businessName : "";
```

- `condition ? A : B` — ternary "if-else in one line".
- "If businessName was sent, use it; otherwise use empty string `""`." (Buyers don't send a business name, so this avoids an error.)

```
$query = "INSERT INTO users
          (account_type, full_name, email, phone, id_card, city,
           postal_code, address, business_name, business_category, user_password)
          VALUES
          ('$accountType', '$fullName', '$email', '$phone', '$idCard', '$city',
           '$postalCode', '$address', '$businessName', '$businessCategory', '$password')";
```

- The SQL command as text.
- `INSERT INTO users (columns...) VALUES (values...)` — add one new row to the `users` table.
- First parentheses = column names. Second = the matching values.
- `'$variable'` — PHP swaps the variable's value into the string (string interpolation). The quotes `'...'` are required because these are text values in SQL.

```
$process_query = mysqli_query($connect, $query);
```

- `mysqli_query($connect, $query)` — actually RUN the SQL on the database.

- For an INSERT it returns `true` if it worked, `false` if it failed. Saved in `$process_query`.

```
if ($process_query) {
    echo json_encode([
        "success" => true,
        "message" => "Account created successfully for $fullName."
    ]);
}
```

- If the insert worked:
- Build a PHP array `["success"=>true, "message"=>"..."]`. (`=>` links a key to its value.)
- `json_encode(...)` — turn it into JSON text.
- `echo` — send it back as the response. This is what axios receives as `response.data`.

```
} else {
    echo json_encode([
        "success" => false,
        "message" => "Try again. Database insertion failed: " . mysqli_error($connect)
    ]);
}
```

- If it failed: send `success:false` and the DB error message (`mysqli_error` = what went wrong).

```
} else {
    echo json_encode([
        "success" => false,
        "message" => "No data was received by the server."
    ]);
}
```

- This `else` belongs to the very first `if (isset...)`. If email/password were missing, say so.

3.2 contact_processing.php

Identical structure. Only differences: - Guard is `if (isset($data->email) && isset($data->message))` — a contact message must have email + message. - `INSERT INTO contacts (full_name, email, phone, subject, message_type, priority, company, message) VALUES (...)` — writes to the **contacts** table instead.

3.3 user_add_processing.php

Identical structure. Differences: - `$fullName = $data->name;` — the admin form field is called `name` (not `fullName`), but the DB column is `full_name`. So we read `->name` and store it in the `full_name` column. - Inserts the extra admin columns: `role`, `status`, `join_date`.

3.4 user_update_processing.php (UPDATE, not INSERT)

```
if (isset($data->id) && isset($data->email)) {
    $id = $data->id;
    // ...other fields...
```

- Needs an `id` so it knows WHICH row to change.

```
$query = "UPDATE users SET
            full_name = '$fullName',
            email     = '$email',
            ...
          WHERE id = '$id'";
```

- `UPDATE users SET column='value', ...` — change columns in EXISTING rows.
- `WHERE id = '$id'` — **only** the row with this id. (Without `WHERE`, it would overwrite EVERY user — dangerous.)

```
if (isset($data->password) && $data->password != "" && $data->password != "********") {
    $password = $data->password;
    $query = "UPDATE users SET ... user_password = '$password' ... WHERE id = '$id'";
}
```

- The edit form pre-fills the password box with `********` (a placeholder, not the real password).
- This says: only include the password in the update **if** the admin actually typed a new one (not empty, not the placeholder). Otherwise keep the old password untouched.
- `!=` means "not equal".

Then the same `mysqli_query` + success/fail JSON as the others.

PART 4 — serviceApi.js (the React side bridge)

```
import axios from "axios";
```

- Load the axios library.

WRITE functions

```
export const process_registration = async (data) => {
  const response = await axios.post(
    "http://localhost/yaqeen-backend/register_processing.php",
    data
  );
  return response.data;
};
```

- `export const process_registration` — a function other files can import.
- `async` — marks it as doing slow network work (lets us use `await`).
- `(data) =>` — it receives the form data object.
- `axios.post(url, data)` — send a POST request to that PHP file, with `data` as the JSON body. axios auto-converts the JS object to JSON.
- `await` — pause until PHP replies (page stays responsive).

- `response` — axios's reply object. `response.data` = the JSON PHP echoed, already parsed back into a JS object.
- `return response.data` — hand it back to whoever called this function.

`process_contact`, `process_user_add`, `process_user_update` are the same, just different URLs.

READ functions (Session 2)

```
export const get_products = async () => {
  const response = await axios.get(
    "http://localhost/yaqeen-backend/get_products.php"
  );
  return response.data;
};
```

- `axios.get(url)` — a GET request (no body; we're asking for data, not sending it).
- Returns the array of products.

```
export const get_product = async (id) => {
  const response = await axios.get(
    "http://localhost/yaqeen-backend/get_product.php?id=" + id
  );
  return response.data;
};
```

- Takes an `id`, glues it to the URL: `...get_product.php?id=3`.
- PHP reads that `3` via `$_GET['id']`.

`get_testimonials`, `get_users` = like `get_products` (no id). `get_user(id)` = like `get_product(id)`.

PART 5 — READ endpoints (Session 2)

5.1 get_products.php

After the CORS block + `include("connection.php")`:

```
$query  = "SELECT * FROM products";
$result = mysqli_query($connect, $query);
```

- `SELECT * FROM products` — "give me all columns (`*`) of all rows in products".
- `mysqli_query` runs it. For a SELECT, `$result` is a **result set** (a cursor pointing at the returned rows), not the data itself yet.

```
$products = [];
while ($row = mysqli_fetch_assoc($result)) {
    $products[] = $row;
}
```

- `$products = []` — start with an empty array.
- `mysqli_fetch_assoc($result)` — pull the NEXT row as an associative array, e.g. `["id"=>"1","title"=>"PS5",...]`.
- Each loop pass grabs the next row; when no rows remain it returns `null`, which is falsy, so the `while` stops.
- `$products[] = $row` — append the row to the array.
- After the loop, `$products` holds every product row.

```
echo json_encode($products);
```

- Convert the PHP array to a JSON string and send it. axios receives this.

5.2 get_product.php (one row)

```
$id = $_GET['id'];
```

- Read the `id` from the URL query string (`?id=3`).

```
$query  = "SELECT * FROM products WHERE id = '$id'";
$result = mysqli_query($connect, $query);
$product = mysqli_fetch_assoc($result);
echo json_encode($product);
```

- `WHERE id = '$id'` — only the matching row.
- One row expected, so `mysqli_fetch_assoc` is called once (no loop) → a single object.
- `json_encode` sends `{"id":"3",...}`.

5.3 get_testimonials.php / get_users.php

Same loop pattern as `get_products.php`.

`get_users.php` has one special trick:

```
$query = "SELECT id, full_name AS name, email, phone, role, status, city, join_date AS
joinDate FROM users";
```

- The DB columns are `full_name` and `join_date`, but the old React code expects `name` and `joinDate`.
- `full_name AS name` — rename the column **in the result** so React's existing code works without changes.

5.4 get_user.php (one row, renamed columns)

Same as `get_product.php` but selects one user and renames columns (`id_card AS idCard`, `postal_code AS postalCode`, etc.) to match the edit form's field names.

PART 6 — The React form wiring (WRITE, Session 1)

Pattern applied to Registration / Contact / UserAdd / UserEdit. Using **Registration.jsx** as the example.

Added imports + state

```
import { useState } from 'react';
import { process_registration } from '../serviceApi';
```

- `useState` — React tool to remember a value between renders.
- `process_registration` — the axios function we wrote.

```
const [serverMessage, setServerMessage] = useState('');
```

- Creates a state variable:
- `serverMessage` = current value (starts empty `''`).
- `setServerMessage` = the function to change it (changing it re-renders the screen).

The submit function

```
async function submit(data) {
```

- `async` — because it will `await` the network.
- `data` — react-hook-form automatically passes the validated form values here.

```
  if (data.password !== data.confirmPassword) { alert('Passwords do not match!'); return; }
  if (!data.agreeTerms) { alert('You must agree to the terms and conditions.'); return; }
  if (data.accountType === 'seller' && !data.businessName) { alert('Business name is
required...'); return; }
```

- Extra manual checks. `!==` = not equal. `return` = stop the function early if a check fails.

```
  try {
    const result = await process_registration(data);
    setServerMessage(result.message);
  } catch (error) {
    setServerMessage('A server error 500 occurred.');
  }
}
```

- `try { ... } catch { ... }` — attempt the risky network call; if it throws, run `catch` instead of crashing.
- `await process_registration(data)` — send the form to PHP, wait for the reply.
- `result` = the JSON object PHP sent (`{success, message}`).
- `setServerMessage(result.message)` — store PHP's message → screen re-renders and shows it.
- `catch` — if the server is down/errored, show a fallback message.

Showing the message

```
{serverMessage && (
  <p className="text-center mt-3 mb-0"><strong>{serverMessage}</strong></p>
)}
```

- `{condition && <jsx>}` — React shortcut: "if `serverMessage` is not empty, render the `<p>`". Empty string is falsy → nothing shows until there's a message.

The bug we fixed (zod schema)

```
agreeTerms: z.boolean().optional(),
```

- The "I agree" checkbox was registered but **missing from the zod schema**. zod **deletes** any field not in the schema, so `data.agreeTerms` was always `undefined` → the check always failed.
- Adding this line keeps the checkbox value in `data`.

UserEdit extra (sends the id)

```
const result = await process_user_update({ ...data, id });
```

- `{ ...data, id }` — copy all form fields (`...data`) AND add `id` (from the URL via `useParams`). PHP needs the id for the `WHERE id = ...` in its UPDATE.

PART 7 — The React page wiring (READ, Session 2)

Pattern applied to all 8 pages. Using **Products.jsx** as the example.

```
import { useState, useEffect } from 'react';
import { get_products } from '../serviceApi';
```

- `useEffect` — run code when the page first appears.

```
const [productsData, setProductsData] = useState([]);
```

- State starts as `[]` (empty array). Important: `.map()` / `.filter()` on `[]` is safe; on `undefined` it would crash before data arrives.

```
useEffect(() => {
  async function load() {
    const data = await get_products();
    setProductsData(data);
  }
  load();
}, []);
```

- `useEffect(fn, [])` — run `fn` **once**, right after first render. The empty `[]` (dependency list) means "never run again".

- Why the inner `async function load()`? The `useEffect` callback itself **cannot** be `async`, so we define an async helper inside and call it. This lets us use `await` (the same style as the form submit functions).
- `await get_products()` — start the fetch and pause until the JSON array comes back.
- `setProductsData(data)` — store it → React re-renders → the list below now has real rows.

```
<span>Rs. {Number(product.price).toLocaleString()}</span>
```

- **Critical detail:** MySQL/PHP returns numbers as **strings** (`"12500"`).
- `"12500".toLocaleString()` does NOT add commas (it's a string method, not a number method).
- `Number("12500")` converts it to the real number `12500`, then `.toLocaleString()` → `"12,500"`.
- That's why every displayed price is wrapped in `Number(...)`.

Per-page differences

Page	What's different
LandingPage.jsx	Two states + two fetches (`get_products` and `get_testimonials`) in one `useEffect`.
ProductDetail.jsx	`const { id } = useParams()` reads the id from the URL. `useState(null)` (one product, starts null). `useEffect(..., [id])` re-fetches if the id changes. While `product` is null it shows "Loading product...".
ProductsList.jsx (admin)	`useState([])` + `useEffect` fetch. Delete button still only removes from local state (not the DB).
UsersList.jsx (admin)	`get_users()` ; the `AS name/joinDate` renames mean the table code is unchanged.
Dashboard.jsx (admin)	Fetches products + users only to count them (`.length`) for the stat cards.
UserEdit.jsx	`get_user(id).then(user => reset({...}))` — `reset` fills every form field with the DB values. Password stays as the `********` placeholder.
ProductEdit.jsx	`get_product(id)` then `reset({...})`. No save (it has an image field → skipped per the rule).

PART 7B — Login (validation, two layers)

Login is special: it does NOT save data — it **reads** the users table to check if the email + password match. Two layers of validation: 1. **zod** (client) — checks the email looks valid and password is long enough. Bad format = it never even submits. 2. **database** (server) — checks the email + password actually exist together in the `users` table.

login_processing.php

After the usual CORS block + `include("connection.php")` + reading the JSON body:

```
if (isset($data->email) && isset($data->password)) {
    $email    = $data->email;
    $password = $data->password;
```

- Make sure both were sent, then copy them out.

```
    $query  = "SELECT * FROM users WHERE email = '$email' AND user_password = '$password'";
    $result = mysqli_query($connect, $query);
```

- `SELECT * FROM users WHERE email = ... AND user_password = ...` — ask the DB: "is there a row with BOTH this email AND this password?".
- `AND` = both conditions must be true.

```
    if (mysqli_num_rows($result) == 1) {
        $user = mysqli_fetch_assoc($result);
        echo json_encode([
            "success" => true,
            "message" => "Login successful. Welcome back, " . $user["full_name"] . ".",
            "role"    => $user["role"],
            "name"    => $user["full_name"]
        ]);
    }
```

- `mysqli_num_rows($result)` — how many rows matched. `== 1` = "exactly one match" = correct credentials.
- Read that row, send back `success:true` plus the user's `role` (so React knows where to send them) and name.

```
    } else {
        echo json_encode(["success" => false, "message" => "Invalid email or password."]);
    }
}
```

- `== 1` was false (0 matches) → wrong email or password → send the failure message.

serviceApi.js — process_login

```
export const process_login = async (data) => {
  const response = await axios.post(
    "http://localhost/yaqeen-backend/login_processing.php",
    data
  );
  return response.data;
};
```

- Same shape as the other POST functions: send the typed email+password, get back `{success, message, role}`.

Login.jsx — the submit

```
async function submit(data) {
  try {
```

```
    const result = await process_login(data);

    if (result.success) {
      login(data.email);
      if (result.role === 'Admin') {
        navigate('/admin');
      } else {
        navigate('/');
      }
    } else {
      setServerMessage(result.message);
    }
  } catch (error) {
    setServerMessage('A server error 500 occurred.');
  }
}
```

- `await process_login(data)` — ask PHP to check the credentials.
- `if (result.success)` — DB found a match:
- `login(data.email)` — store the email in `AuthContext` (marks the user as logged in).
- `if (result.role === 'Admin')` — send admins to `/admin`, everyone else to the home page `/`.
- **Important:** the redirect uses the `role` from the **database**, not the dropdown on the form — so you can't fake being admin by picking it in the menu.
- `else` — no match → show the red "Invalid email or password" under the form.
- `catch` — server down/errored → fallback message.

A note on the read style (sir's level)

The page fetches use an inner `async function load() { ... } load();` inside `useEffect`. An earlier version used `.then()` plus an `Array.isArray()` safety guard, but those were removed to stay exactly at the taught toolbox (`useEffect` + `await`, same as the form submits). Trade-off: if you open a page **before** MySQL is running, it errors instead of showing an empty list — so always start XAMPP MySQL first.

Why `Number(product.price)`?

MySQL/PHP sends numbers back as **text strings** (`"12500"`). The original code called `price.toLocaleString()` to add the thousands commas, but that only works on a real number. `Number("12500")` turns the string into the number `12500`, then `.toLocaleString()` → `"12,500"`. It's plain JavaScript, kept only so the prices still show commas like the original static version.

PART 8 — The database (SQL)

users table (Session 1)

```
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    account_type VARCHAR(20), full_name VARCHAR(100), email VARCHAR(100),
```

```
    ...
);
```

- `CREATE TABLE users (...)` — make a new table.
- `id INT AUTO_INCREMENT PRIMARY KEY` — a number id that the DB fills in automatically (1,2,3...) and that uniquely identifies each row.
- `VARCHAR(100)` — a text column up to 100 characters. `TEXT` = longer text. `INT` = whole number. `DECIMAL(2,1)` = number like 4.8.

contacts table + ALTER (Session 1)

```
ALTER TABLE users
    ADD COLUMN role VARCHAR(20),
    ADD COLUMN status VARCHAR(20),
    ADD COLUMN join_date VARCHAR(20);
```

- `ALTER TABLE users ADD COLUMN ...` — add new columns to an EXISTING table (the admin user forms needed role/status/join_date).

products + testimonials tables + seed (Session 2)

```
TRUNCATE TABLE products;
INSERT INTO products (id, title, price, ...) VALUES (1, 'PS5...', 12500, ...), (2, ...), ...;
```

- `TRUNCATE TABLE products` — empty the table first (so re-running doesn't duplicate).
- One big `INSERT ... VALUES (...),(...),(...)` — add many rows at once.
- `\`condition\`` and `\`text\`` are wrapped in backticks because those words are reserved in MySQL.

run_seed.php (the seeder)

```
$sql = file_get_contents("seed_data.sql");
if (mysqli_multi_query($connect, $sql)) { ... }
```

- `file_get_contents("seed_data.sql")` — read the whole SQL file into a string.
- `mysqli_multi_query` — run MANY SQL statements separated by `;` in one go (normal `mysqli_query` only runs one).
- Used because `seed_data.sql` has many `CREATE` / `INSERT` statements.

PART 9 — Full request lifecycle (putting it together)

Example: opening the Products page.

1. Browser loads `Products.jsx`. State `productsData = []`. Page briefly shows "0 products".
2. `useEffect` fires once → calls `get_products()`.
3. `get_products` runs `axios.get("http://localhost/yaqeen-backend/get_products.php")`.

4. Apache runs `get_products.php`:
5. CORS headers sent.
6. `include connection.php` → opens MySQL.
7. `SELECT * FROM products` → MySQL returns 8 rows.
8. loop builds a PHP array → `json_encode` → `echo` sends JSON text.
9. axios receives the JSON, parses it to a JS array, returns it as `response.data`.
10. `.then((data) => setProductsData(data))` stores the array in state.
11. State change → React re-renders → `.filter().map()` draws 8 product cards with `Number(price)` formatted.

Example: submitting the Registration form.

1. User fills form → clicks Create Account.
2. react-hook-form + zod validate. If invalid, errors show and submit never runs.
3. `submit(data)` runs → `await process_registration(data)`.
4. `axios.post(url, data)` sends JSON to `register_processing.php`.
5. PHP: read body → `json_decode` → `INSERT INTO users ...` → MySQL adds a row → `echo {success, message}`.
6. axios returns it → `setServerMessage(result.message)` → green message shows under the form.
7. The new row is visible in phpMyAdmin (and on the admin Users list, which reads from the same table).

PART 10 — Vocabulary cheat-sheet

Term	Meaning
`axios.get / .post`	send an HTTP request from React
`useEffect(fn, [])`	run `fn` once when the page loads
`useState(x)`	remember a value; changing it re-renders
`.then(...)`	"when the promise finishes, do this"
`await`	pause until the async thing finishes
`include` (PHP)	paste another PHP file's code here
`$_GET['id']`	read a value from the URL (`?id=...`)
`php://input`	the raw POST body (the JSON axios sent)
`json_decode` / `json_encode`	JSON text → PHP object / PHP → JSON text
`mysqli_query`	run one SQL command
`mysqli_fetch_assoc`	get the next row as an array
`INSERT / SELECT / UPDATE`	add / read / change rows

Term	Meaning
`WHERE id = ...`	target a specific row
`AS name`	rename a column in the result
CORS headers	let a different port call this PHP
`Number(...)`	turn a string like "12500" into the number 12500
``	